

为什么我还是无法理解transformer

你脑子没问题，是那些教程有问题。

Transformer的训练方式跟你理解的**神经网络**一模一样，就是**反向传播**，就是调整权重参数，没有任何新东西。

你之所以困惑，是因为99%的教程犯了一个致命错误：它们花大量篇幅讲**注意力机制**的前向传播过程，把Q、K、V的矩阵运算讲得天花乱坠，然后到了训练部分就一笔带过甚至直接跳过。这给人一种错觉，好像Transformer有什么神秘的训练方法。

没有。真的没有。

你的困惑根源是把前向传播和训练机制搞混了。我先帮你理清一个概念上的混乱。

当人们讲Transformer的时候，讲的是它的网络结构和前向传播的计算流程。Q、K、V那套公式，Softmax加权求和，多头注意力，这些东西描述的是数据怎么从输入流到输出。这是前向传播。

而你问的反向传播怎么调参数，这是训练机制。

关键来了：训练机制跟网络结构是两回事。不管你的网络结构长什么样，只要它是由可微分的操作组成的，反向传播就能用。Transformer里面的每一个操作，矩阵乘法、Softmax、**LayerNorm**、**残差连接**，全都是可微分的。所以反向传播直接就能用，跟训练一个普通的三层全连接网络没有本质区别。

你之前理解的那套东西，损失函数对参数求偏导，链式法则一层层往回传，梯度下降更新权重，这套东西在Transformer里面一字不改，直接照搬。

所以你的问题其实不应该是Transformer怎么反向传播，而应该是：Transformer里面到底有哪些参数是需要训练的？

这才是你真正没搞清楚的地方。

可训练参数

Transformer里到底有哪些可训练参数？

第一块：**词嵌入层** Embedding

输入是一串token ID，比如 [101, 2769, 3221, 4638, 102]。词嵌入层是一个查找表，维度是 词表大小 × 嵌入维度，比如 50000 × 768。这整个嵌入矩阵就是可训练参数，跟你训练word2vec没区别。

第二块：**位置编码** Positional Encoding

原始论文用的是正弦余弦函数算出来的固定值，不需要训练。但现在很多模型用的是可学习的位置嵌入，比如**BERT**和**GPT系列**，这又是一个 最大序列长度 × 嵌入维度 的矩阵，也是可训练参数。

第三块：**自注意力层**，这是重点

这里是大家讲Q、K、V的地方，但大多数教程只讲了公式没讲参数。我给你说清楚。

Q、K、V是怎么来的？是输入X分别乘以三个权重矩阵得到的：

$$Q = X \times W_q$$

$$K = X \times W_k$$

$$V = X \times W_v$$

W_q 、 W_k 、 W_v 这三个矩阵就是可训练参数。假设嵌入维度是768，那每个矩阵就是 768 × 768 的，三个加起来就是将近180万个参数。

然后多头注意力算完之后，还要再乘一个输出投影矩阵 W_o ，又是 768×768 的参数。

所以一个自注意力层里面，光权重矩阵就有四个： W_q 、 W_k 、 W_v 、 W_o 。这就是需要训练的东西。

第四块：前馈神经网络 FFN

每个Transformer层里面，注意力算完之后还要过一个两层的前馈网络。标准配置是先升维到 $768 \times 4 = 3072$ ，过一个激活函数，再降回 768。

这两层的权重矩阵分别是 768×3072 和 3072×768 ，外加两个偏置向量。参数量比注意力层还大，差不多470万个参数。

第五块：LayerNorm层

每个子层后面都有一个Layer Normalization，它有两个可训练参数向量，一个是缩放系数 γ ，一个是偏移系数 β ，每个都是768维。参数量不大，但也是要训练的。

好，我帮你算一下。一个标准的Transformer层，包含一个多头自注意力和一个FFN，参数量大概是：

- 注意力部分： $768 \times 768 \times 4 =$ 约240万
- FFN部分： $768 \times 3072 \times 2 =$ 约470万
- LayerNorm： $768 \times 4 =$ 约3000

一层就是700多万参数。BERT-base有12层，光这部分就是8000多万。再加上词嵌入层的3800万，总共1.1亿参数左右，跟官方公布的110M对得上。

我把这些参数展开了讲，就是想让你明白：**Transformer不是什么玄学，它就是一堆矩阵乘法和非线性变换堆起来的，每个矩阵都是可训练参数。**

反向传播

反向传播的时候，损失函数对这些参数求梯度，梯度下降更新参数，跟你训练一个最简单的MLP没有任何本质区别。PyTorch的autograd或者TensorFlow的GradientTape帮你自动算好了所有梯度，你甚至不需要手推公式。

为什么教程都不讲这些？因为讲这些没啥可讲的。

你想想，一个教程作者写Transformer介绍，如果训练部分写的是这就是普通的反向传播没什么特别的，读者会觉得你在水字数。所以大家都把篇幅花在注意力机制的原理上，那玩意儿看起来高大上，有公式有图，写起来也有东西可写。

但这就造成了一个信息缺口：初学者看完之后知道Transformer的前向传播是怎么算的，但对于它为什么能训练、怎么训练完全没概念。

还有一个原因是，写教程的人默认你已经懂反向传播了。他们觉得既然你知道神经网络怎么训练，那Transformer作为神经网络的一种，你自然知道它也是一样训练的。但他们没想到，正是因为Transformer的前向传播过程太特别了，注意力机制太新颖了，反而让初学者产生了它的训练方式也应该很特别的错觉。

我早年带新人的时候就发现这个问题。有个刚毕业的小伙子，NLP背景很好，LSTM什么的门清，但看了一周Transformer愣是没看懂。后来我问他具体哪里不懂，他说的跟你一模一样：反向传播怎么做的，参数怎么更新的，那些教程都没讲。

我当时给他的回答也是这个：没什么特别的，就是普通的反向传播。他说不可能吧，那个注意力权重是动态算出来的，不是固定的参数，这怎么反向传播？

需要注意的是：注意力权重不是参数，但它是参数的函数。这是很多人混淆的地方。

$\text{Softmax}(QK^T / \sqrt{d})$ 这个东西算出来的注意力权重矩阵，确实不是直接的可训练参数。它是输入X经过Wq和Wk变换之后算出来的中间结果。每次输入不同的句子，这个注意力矩阵都是不一样的。

但这不影响反向传播。

反向传播的核心是链式法则。损失函数L对某个参数 θ 的梯度是：

$$\partial L / \partial \theta = \partial L / \partial \text{输出} \times \partial \text{输出} / \partial \text{中间变量} \times \dots \times \partial \text{某个中间变量} / \partial \theta$$

注意力权重虽然是动态计算的，但它是Wq、Wk这些参数的函数。只要这个函数是可微分的，梯度就能沿着计算图一路传回去。

Softmax可微，矩阵乘法可微，除法可微，所以整条链路都是通的。

具体来说，反向传播的时候，梯度会这样流动：

损失 → 模型输出 → 最后一层FFN → 最后一层注意力输出 → W_o → $V \times$ 注意力权重 → Softmax → QK^T → Q和K → Wq和Wk → 输入X → ... → 一直到词嵌入层

每一步都是标准的矩阵求导。这里面唯一稍微复杂一点的是Softmax的导数，但那也是有解析形式的。如果你对矩阵变换和链式法则的几何直觉还不够清晰，去看 3Blue1Brown 的《线性代数的本质》和《微积分的本质》。那个动画演示能让你亲眼看到矩阵是如何扭曲空间的，梯度又是如何沿着函数曲面下滑的。Transformer 里那些看似吓人的公式，拆解到底层就是这些最基础的几何变换。[3Blue1Brown 线性代数笔记：可能是全网最好的中英文整理](#)

如果你想亲手验证这件事，最好的办法是自己用numpy从零实现一遍。网上有一份代码我觉得写得挺清楚的，是哈佛NLP组做的 The Annotated Transformer，用PyTorch从零实现了整个Transformer，每一行代码都有注释。你跑一遍，打印一下每个参数的梯度，就彻底明白了。

还有一个被圈内人称为保姆级的神作——李沐老师的《动手学深度学习》(Dive into Deep Learning) [《动手学深度学习》2.0版本中英文+在线电子书](#)。这本书最大的好处是所见即所得，理论旁边直接放着可运行的 PyTorch 代码。你看着公式如果发懵，直接点一下运行，看数据在代码里是怎么流转的，立马就懂了。

训练过程

我给你还原一个真实的训练过程。

光讲原理可能还是有点抽象，我给你描述一下实际训练一个Transformer模型是什么体验。

假设你要训练一个做文本分类的BERT模型，数据集是中文情感分析，二分类任务。

第一步，准备数据

你有一堆句子和对应的标签，正面是1负面是0。分词之后转成token ID，加上特殊标记 [CLS] 和 [SEP]，padding到统一长度，打包成batch。

第二步，前向传播

一个batch的数据喂进模型。词嵌入层把token ID转成向量，加上位置编码。然后进入12层Transformer encoder，每一层做自注意力和FFN，中间有残差连接和LayerNorm。最后取 [CLS] 位置的输出向量，过一个分类头得到二分类的logits。

这一步就是所有教程在讲的东西。

第三步，算损失

logits和真实标签算交叉熵损失，得到一个标量loss。

第四步，反向传播

调用 `loss.backward()`，PyTorch自动帮你算所有参数的梯度。这一步在代码里就一行，背后的计算图遍历、链式法则应用、梯度累加，全是框架帮你做的。

第五步，更新参数

调用 `optimizer.step()`，根据梯度更新所有参数。用的优化器一般是AdamW，学习率调度用warmup加线性衰减。

第六步，清空梯度，进入下一个batch

调用 `optimizer.zero_grad()`，然后重复第二步到第五步。

你看，整个流程跟训练一个简单的CNN分类器一模一样。区别只在于模型结构不同，参数量不同，其他流程完全相同。

我2019年第一次从零训练一个Transformer模型的时候，最大的感受就是这个东西跟我训练LSTM没啥区别，除了显存占用大了一个数量级、训练时间长了好几倍之外，代码模式完全一样。

我再多说一点，帮你建立一个更完整的认知。

很多人以为Transformer的核心创新是注意力机制。其实不完全对。

注意力机制这个概念在Transformer之前就有了，Bahdanau在2014年的seq2seq论文里就提出了。那时候的注意力是用在encoder-decoder架构里，让decoder在生成每个词的时候能动态关注encoder输出的不同位置。

Transformer的真正创新是什么呢？是用注意力完全替代了循环结构。

在Transformer之前，处理序列数据的标准范式是RNN/LSTM/GRU这类循环神经网络。这些模型有个致命问题：序列处理是串行的，第t步的计算依赖于第t-1步的隐状态，没法并行。这导致训练速度很慢，也没法充分利用GPU的并行计算能力。

Transformer把循环结构干掉了，整个序列的所有位置可以同时计算。一个1024长度的序列，在RNN里要算1024步，在Transformer里自注意力一次矩阵乘法就搞定了。这才是它能scale up到现在这个规模的根本原因。

所以Transformer的革命性不在于注意力机制本身有多神奇，而在于它证明了：不用循环结构也能处理序列，而且效果更好。这打开了一个新世界的大门。

如果你想深入理解这一点，可以去看原始论文 [Attention Is All You Need](#)。这篇文章写得非常清楚，没有故弄玄虚。特别是第三节 Model Architecture 和第四节 Why Self-Attention，把设计动机讲得很透彻。

既然聊到这了，我顺便讲讲Q、K、V这个设计的直觉是什么，帮你从另一个角度理解。

自注意力的核心操作是：对于序列中的每个位置，计算它应该关注其他哪些位置，然后把信息聚合起来。

最朴素的做法是直接原始向量算相似度，两个位置的向量点积大，就说明它们相关性高，应该多关注。但这样太简单了，表达能力不够。

Q、K、V的设计引入了三个独立的线性变换，相当于让模型学习三个不同的投影空间：Query空间、Key空间、Value空间。

Query代表我要查询什么信息，Key代表我能提供什么信息用于匹配，Value代表我实际能提供的内容。Query和Key在各自空间里算相似度，然后用这个相似度去加权Value。

这三个空间是独立学习的，给了模型更大的灵活性。它可以学到：某两个词在语义上相关（Query-Key匹配分高），但提取的特征不同（各自的Value不同）。

多头注意力更进一步，有8个或者16个独立的头，每个头学习不同的关注模式。有的头可能专注于语法关系，有的头专注于语义关系，有的头专注于位置距离。最后把所有头的结果拼起来，再投影回原始维度。

这个设计非常优雅，而且确实work。后来的研究发现，不同的头真的学到了不同类型的attention pattern。你可以用一些可视化工具看看，比如 BertViz 这个项目，把预训练BERT的注意力权重可视化出来，能看到很有意思的规律。

但说到底，这些只是模型结构的设计选择。训练的时候， W_q 、 W_k 、 W_v 这些矩阵就是普通的参数，用反向传播更新，跟其他参数没有区别。

我一直觉得，对于Transformer这种稍微复杂一点的模型，光看教程是不够的，必须自己动手写一遍。

不是说你要从零写一个工业级的实现，那没必要也没意义。但是用numpy或者纯PyTorch手写一个最简版本，把forward和backward都过一遍，对于理解模型结构有巨大的帮助。

我推荐两个资源：

一个是前面提到的 The Annotated Transformer，哈佛NLP组出的，用PyTorch实现了原始论文的所有细节，代码和注释对照着看非常清晰。这个适合你想完整理解Transformer架构的情况。

另一个是Andrej Karpathy的 minGPT 和 nanoGPT 项目，代码极其简洁，实现了一个最小化的GPT模型。如果你想快速上手玩起来，这个是最佳选择。Karpathy还配了一个两小时的视频教程叫 Let's build GPT: from scratch, in code, spelled out，讲得非常好，我看完之后对于GPT系列的理解提升了一大截。

你可以按这个顺序来：先跟着Karpathy的视频把nanoGPT跑通，理解decoder-only的架构。然后看The Annotated Transformer，理解encoder-decoder的完整架构。最后如果有兴趣，去看看HuggingFace的transformers库源码，看看工业级实现是怎么处理各种细节的。

这三步走下来，Transformer对你来说就不再神秘了。

到这一步，如果你已经理解了基础的训练机制，接下来可以关注一些进阶问题：

大模型的训练稳定性

模型大了之后训练会变得不稳定，loss容易spike甚至nan。这时候需要一些trick：Pre-LayerNorm而不是Post-LayerNorm、参数初始化的scale要根据层数调整、学习率warmup、梯度裁剪等等。这些不是什么神秘的黑魔法，都是工程上踩坑踩出来的经验。

显存优化

Transformer的显存占用跟序列长度的平方成正比，因为要存注意力矩阵。长序列训练时显存很容易爆。解决方案包括梯度检查点（用计算换显存）、Flash Attention（通过kernel fusion和tiling减少显存访问）、混合精度训练等。这些是做大模型训练必须要了解的技术。

各种高效注意力变体

标准自注意力的计算复杂度是 $O(n^2)$ ，对于长序列不友好。所以有很多改进版本：Sparse Attention只计算部分位置对、Linear Attention把复杂度降到 $O(n)$ 、Sliding Window Attention只看局部窗口等。这些变体在特定场景下有用，但也有各自的trade-off。

位置编码的演进

原始Transformer用的是固定的正弦位置编码，后来BERT改成了可学习的位置编码。再后来ALiBi用位置偏移代替位置编码，RoPE用旋转矩阵编码相对位置。这些改进主要是为了解决外推问题，让模型能处理比训练时更长的序列。

这些进阶话题每一个都可以展开很多，但都不改变一个基本事实：Transformer就是一个神经网络，用反向传播训练，跟你之前学的东西是一脉相承的。

回到你的问题，你说看了很多资料还是不理解Transformer。我觉得你其实已经理解了最核心的东西，就是神经网络通过反向传播调整参数来拟合数据。Transformer也是这样的。

你之所以觉得不理解，是因为教程们给你制造了一个错误的期待：你以为Transformer有什么特殊的训练方式，但其实没有。

Q、K、V的公式只是描述了注意力计算的前向过程，告诉你数据怎么从输入变成输出。训练的时候， W_q 、 W_k 、 W_v 这些矩阵跟普通的全连接层权重一样，用反向传播算梯度，用优化器更新参数，nothing special。

如果你还想要一个更形象的理解：可以把Transformer想象成一个超大号的矩阵乘法堆叠，中间夹着一些非线性函数。数据从左边进去，经过一堆矩阵变换，从右边出来。训练就是调整这些矩阵里的数字，让输出尽可能接近你想要的结果。

所以总结一下：

Transformer的训练机制跟普通神经网络没有区别，就是反向传播加梯度下降。你觉得不理解，是因为大多数教程只讲前向传播不讲训练，造成了信息缺口。

Transformer里的可训练参数包括：词嵌入矩阵、位置编码（如果是可学习的话）、每一层的 $W_q/W_k/W_v/W_o$ 、FFN的权重偏置、LayerNorm的参数。这些参数的梯度通过链式法则计算，跟任何其他可微网络一样。

想真正理解Transformer，最好的办法是自己动手实现一遍。推荐Karpathy的nanoGPT和哈佛的The Annotated Transformer，代码都很清晰。

不要被注意力机制的新颖性吓到，它只是一种计算方式，不改变神经网络训练的基本范式。你之前积累的所有关于神经网络的知识，在这里全都用得上。

就这样。去写代码吧。